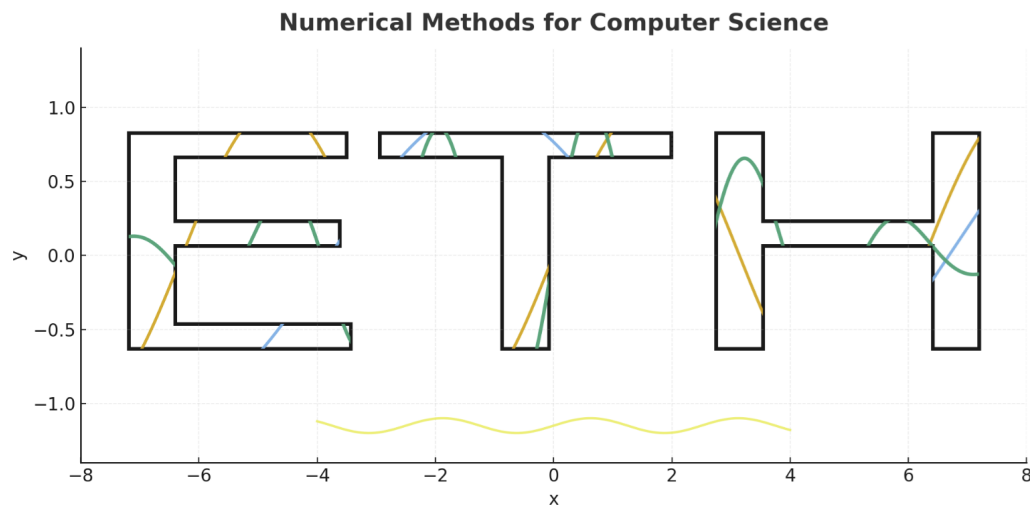




# Trigonometric Interpolation    Fourier Basics — Week #4

ETH Zurich

DAVID Mihnea-Ştefan

mihdavid@ethz.ch

This week introduces trigonometric interpolation as a natural tool for periodic data. We build the bridge from Fourier series to the discrete Fourier transform (DFT), with clear intuition, minimal prerequisites, and practical examples.

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Why Trigonometric Interpolation</b>                            | <b>1</b>  |
| 1.1      | Context and motivation . . . . .                                  | 1         |
| 1.2      | Limitations of global polynomials (Runge) . . . . .               | 1         |
| 1.3      | Periodic data and the trigonometric model . . . . .               | 2         |
| 1.4      | Sampling on a uniform grid . . . . .                              | 3         |
| 1.5      | Why uniform grids work well. . . . .                              | 6         |
| 1.6      | Practical impact. . . . .                                         | 8         |
| <b>2</b> | <b>Fourier Series Basics</b>                                      | <b>9</b>  |
| 2.1      | Why sines and cosines? . . . . .                                  | 9         |
| 2.2      | Inner product and orthogonality (clearer view) . . . . .          | 9         |
| 2.3      | Definition: Fourier series and coefficients (real form) . . . . . | 10        |
| 2.4      | Complex form of the Fourier series . . . . .                      | 11        |
| <b>3</b> | <b>Discrete Fourier Transform (DFT)</b>                           | <b>13</b> |
| 3.1      | From continuous to discrete . . . . .                             | 13        |
| 3.2      | Trigonometric interpolation viewpoint . . . . .                   | 13        |
| 3.3      | Roots of unity and orthogonality . . . . .                        | 13        |
| 3.4      | Fourier matrix and the DFT . . . . .                              | 15        |
| 3.5      | Properties and impact . . . . .                                   | 17        |
| 3.6      | Why is the DFT useful? . . . . .                                  | 17        |

# 1 Why Trigonometric Interpolation

## 1.1 Context and motivation

In numerical methods we often reconstruct a function from samples. The classical tool is *polynomial interpolation*: build a single  $P_n(x)$  that passes through all data points. This is elegant and powerful, yet on large intervals or for oscillatory data it can misbehave.

### Note

#### Big picture.

- **Polynomial interpolation:** works on a finite interval  $[a, b]$  (line). The interval has two endpoints, and the interpolant often develops oscillations near them (Runge phenomenon).
- **Trigonometric interpolation:** works on a periodic interval, e.g.  $[0, 1)$  (circle). The left and right endpoints are identified ( $0 \equiv 1$ ). There are no boundary points, so the interpolant wraps around smoothly.

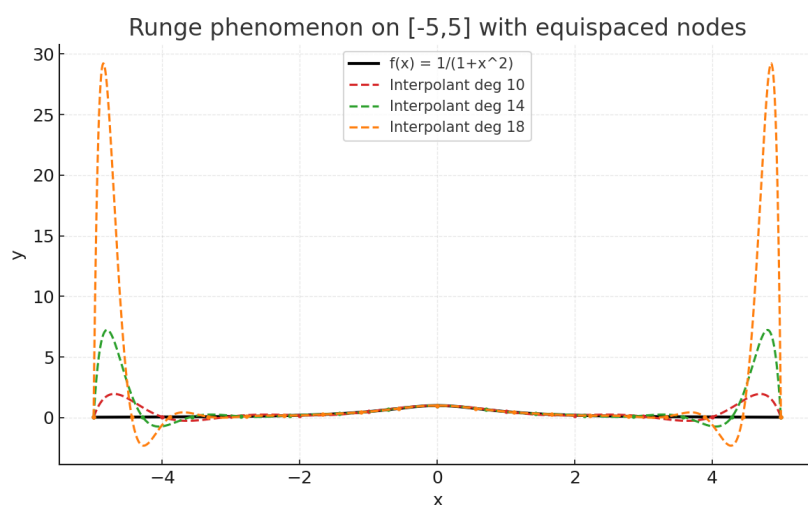
*Key idea: finite interval with edges vs. periodic interval with no edges. This explains why trigonometric interpolation is more stable for periodic data.*

## 1.2 Limitations of global polynomials (Runge)

Consider  $f(x) = \frac{1}{1+x^2}$  on  $[-5, 5]$ . Interpolating on *equispaced* nodes and increasing  $n$  yields:

- exact agreement at the nodes;
- growing oscillations near  $x = \pm 5$  (overshoot/undershoot);
- overall error that can increase with degree.

This is the **Runge phenomenon**. It is structural (not a coding error).



**Why does it happen?** A high-degree polynomial has very limited flexibility: once you fix its values at many points, its shape is determined everywhere. On a long interval this creates tension:

- In the middle the polynomial tries to follow the smooth curve of  $f(x)$ .
- At the edges it is “pulled” up and down to also pass exactly through the endpoints.
- The result is large oscillations near the boundaries, even though the original function is smooth there.

Another way to see it: polynomials of high degree involve large powers of  $x$ . On  $[-5, 5]$  these powers can take very large values, which makes the coefficients of the interpolant unstable and sensitive to tiny changes in the data or in the nodes.

### 1.3 Periodic data and the trigonometric model

Many signals are naturally periodic (or can be analysed on periodic windows): audio, vibrations, rotations, textures, seasonal time series.

A *trigonometric polynomial* of degree  $m$  can be written in the **real form**:

$$p_m(t) = \frac{a_0}{2} + \sum_{k=1}^m \left( a_k \cos(2\pi kt) + b_k \sin(2\pi kt) \right), \quad t \in [0, 1).$$

It is periodic by construction, so the endpoints match:  $t = 0 \equiv 1$ .

The same object can also be written in the **complex exponential form**:

$$p_m(t) = \sum_{k=-m}^m c_k e^{2\pi i k t},$$

where the coefficients  $c_k$  are in general complex numbers.

#### Note

**Equivalence of the two formulas.** Euler’s identities

$$\cos(\theta) = \frac{1}{2} (e^{i\theta} + e^{-i\theta}), \quad \sin(\theta) = \frac{1}{2i} (e^{i\theta} - e^{-i\theta})$$

show that each cosine and sine can be written as a combination of exponentials. Therefore:

- the real form with  $(a_k, b_k)$  coefficients is equivalent to
- the complex form with symmetric coefficients  $c_{-k} = \overline{c_k}$  (so that  $p_m(t)$  is real-valued).

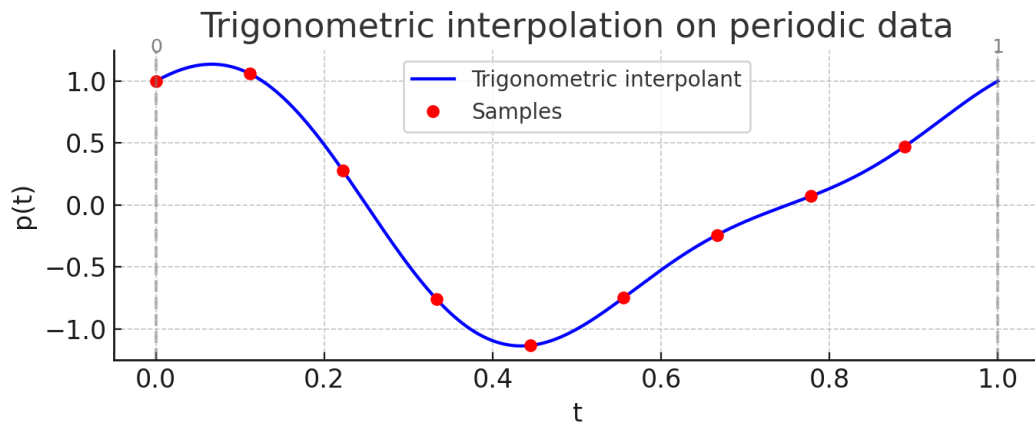


Figure 1: Periodic samples (dots) on  $[0, 1)$  and a smooth trigonometric interpolant (curve) wrapping at  $0 \equiv 1$ .

#### Note

**Interpretability.** Coefficients  $(a_k, b_k)$  measure how much of each frequency is present. This enables frequency-domain tasks (filtering, denoising, compression) directly.

### 1.4 Sampling on a uniform grid

In real life, we do not know a function everywhere. We only measure it at discrete points.

A very common choice is to take *equally spaced samples*:

$$t_j = \frac{j}{N}, \quad j = 0, 1, \dots, N-1,$$

so we obtain  $N$  values

$$y_j = f(t_j).$$

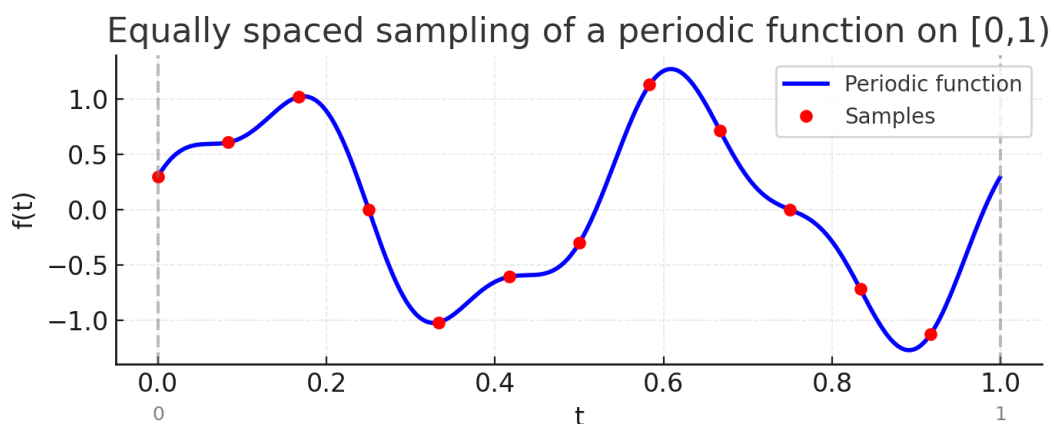


Figure 2: Equally spaced sampling of a periodic function on  $[0, 1)$ .

**How many waves can we capture?** If we have  $N$  sample points, we cannot hope to reconstruct infinitely many frequencies. In fact, only about **half as many frequencies as samples** can be represented uniquely. This rule is called the **Nyquist limit**.

#### Note

##### Nyquist limit (expanded intuition + formula).

*Intuition: the wheel analogy.* Imagine a wheel spinning very fast, and you try to capture it with a camera. If the camera takes many pictures per second, each spoke is tracked correctly and you see the real speed.

But if the camera takes only a few pictures per second, strange things happen:

- The wheel may look like it spins more slowly than it really does.
- Sometimes it may even appear to spin backwards.

This happens because the snapshots are too sparse to distinguish one fast revolution from a slower motion. The camera is *aliasing* a high frequency (fast rotation) as a lower frequency (slow or reversed rotation).

*Formula with sines and cosines.* On  $N$  uniform sample points

$$t_j = \frac{j}{N}, \quad j = 0, 1, \dots, N-1,$$

the sampled values of high-frequency sines and cosines repeat those of lower frequencies. In fact:

$$\cos(2\pi(N-k)t_j) = \cos(2\pi k t_j), \quad \sin(2\pi(N-k)t_j) = -\sin(2\pi k t_j).$$

So the frequencies  $k$  and  $N-k$  cannot be distinguished once we only look at the  $N$  samples.

**Conclusion.** With  $N$  samples we can uniquely represent only frequencies up to  $N/2$ . Beyond that, the “fast wheel” is mistaken for a “slower wheel” — this is aliasing. This threshold is the *Nyquist limit*.

#### What does this mean for interpolation?

- **How much we can represent.** If we have  $N$  equally spaced samples, then the richest trigonometric interpolant we can build has degree at most

$$m = \left\lfloor \frac{N-1}{2} \right\rfloor.$$

In other words:  $N$  points are enough to recover at most  $N/2$  distinct frequency components.

- **Exact recovery of the samples.** With that polynomial we can match all the sample values  $y_j = f(t_j)$  exactly. The interpolant passes through every dot, and for periodic smooth data it also gives a very good approximation between the dots.
- **What if the signal has higher frequencies? (aliasing)** This is the most important

point. If the true signal oscillates faster than the Nyquist limit ( $N/2$ ), then the samples cannot tell it apart from a slower signal. The “excess” frequency is *folded down* into a lower one.

#### Note

**Example.** Suppose we take  $N = 8$  samples. A cosine with frequency  $k = 7$  and one with frequency  $k = 1$  will have identical values at those 8 sample points. From the sampled data alone, they are indistinguishable. This folding effect is what we call *aliasing*.

#### Note

##### Two sources of bias in sampling.

When we sample on  $N$  equally spaced points, two different effects can distort the interpretation of frequency:

1. **Periodicity modulo  $N$ .** Any frequency  $k$  is indistinguishable from  $k + N, k + 2N, \dots$ . This means that, in theory, very high frequencies can be folded back into the range  $[0, N)$ . In practice, however, most signals are band-limited, so we rarely encounter such extremely high multiples.
2. **Folding around Nyquist ( $N/2$ ).** Even within  $[0, N)$ , the frequencies  $k$  and  $N - k$  cannot be distinguished: they give the same cosine values (and opposite sine values) on the sample grid. This is the true aliasing effect: high-frequency components above  $N/2$  are misinterpreted as lower ones (any lower one could be contaminated by a hidden higher one).

**Which is more dangerous?** The first effect (periodicity) is mostly a theoretical curiosity: it requires unrealistically large frequencies. The second effect (aliasing at Nyquist) is *much more serious in practice*, because it contaminates the frequency band of interest with false components.

This is why all real-world systems use *anti-aliasing filters*: low-pass filters that remove frequencies above Nyquist before sampling.

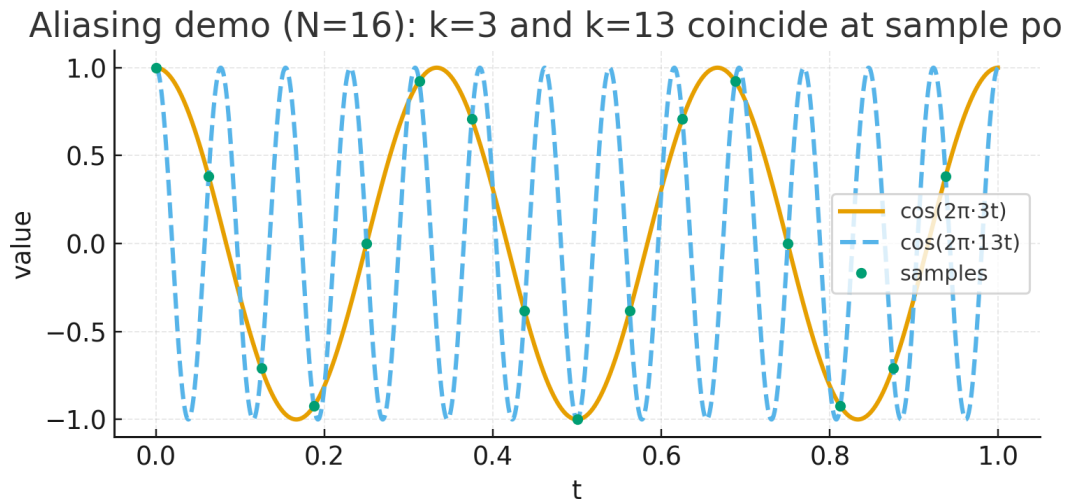


Figure 3: Aliasing (correct example): for  $N = 16$ , the frequencies  $k = 3$  and  $k = 13 = N - k$  produce exactly the same values at the sampling points  $t_j = j/N$ , even though their continuous curves differ between the samples.

#### Note

##### How to read the figure.

The continuous curves show two different cosine waves: one with frequency  $k = 3$  and one with frequency  $k = 13$  (since  $N = 16$ , they form a Nyquist pair). Although the curves differ between the samples, at every sample point  $t_j = j/N$  the two waves have exactly the same value.

This means: if you only see the discrete samples (black dots), you cannot tell whether they came from the low-frequency wave ( $k = 3$ ) or the high-frequency wave ( $k = 13$ ). Both explanations fit the data equally well.

**Key insight:** aliasing creates ambiguity. Once frequencies fold beyond Nyquist, multiple different signals can explain the same sampled points, and there is no way to know which one was the original without extra information.

### 1.5 Why uniform grids work well.

When we sample on a *uniform grid*, the points are equally spaced:

$$t_j = \frac{j}{N}, \quad j = 0, 1, \dots, N-1.$$

This regular spacing gives a huge advantage: every basic wave (sine or cosine of a certain frequency) is sampled in a perfectly balanced way.

Think of it like this:

- A wave of frequency  $k$  will “wiggle” a certain number of times across the  $N$  points.



- Another wave of a different frequency  $m$  will “wobble” a different number of times.
- Because the grid is uniform, the ups and downs of wave  $k$  do not systematically line up with those of wave  $m$ .
- Over the whole grid, the positive parts cancel the negative parts, so the two waves do not overlap in average.

#### Note

**Geometric picture.** If you plot the complex exponential  $e^{2\pi i k t_j}$  as vectors in the plane, they lie equally spaced on the unit circle. For one frequency  $k$ , they all point in a consistent direction; for another frequency  $m$ , they spin around at a different pace. When you add them up, the vectors of one frequency “average out” to zero relative to the other. That is what orthogonality means in this context.

**From uniform grids to a solvable system.** Uniform sampling does more than just give us values of  $f(t)$ . It creates a structured system: each sample is a mixture of contributions from different frequencies, and the uniform grid ensures these contributions are arranged in a predictable way.

Mathematically, we can write

$$y_j = \sum_k c_k e^{2\pi i k t_j}, \quad j = 0, 1, \dots, N-1,$$

where the coefficients  $c_k$  describe the strength (power) of each frequency in the signal. Because of the uniform grid, this system can be solved uniquely. This is exactly what the Discrete Fourier Transform (DFT) does.

**Simplified example (for intuition only).** Suppose  $N = 6$ . Then waves with periods 1, 2, 3, and 6 all align at  $x = 6$ :

- the wave with period 1 completes 6 oscillations,
- the wave with period 2 completes 3 oscillations,
- the wave with period 3 completes 2 oscillations,
- the wave with period 6 completes 1 oscillation.

At that point, they all return to their starting position simultaneously.

This toy example shows why the uniform grid provides natural synchronization points: many different frequencies fit neatly on the same grid, so their contributions can be separated systematically. It is a simplified picture of the more general fact that the Fourier basis functions are orthogonal on a uniform grid.

### 1.6 Practical impact.

The shift from polynomials to trigonometric series changed science and engineering in a fundamental way. Fourier analysis is not only about fitting functions — it is about *signals and frequencies*.

Every audio clip, every image, every vibration can be described as a mixture of frequencies. Once this was understood, new tools became possible:

- audio compression (e.g. MP3) works by keeping only the important frequencies for human hearing,
- image formats (e.g. JPEG) compress data by storing the dominant frequency patterns in blocks of pixels,
- real-time signal processing (filters, equalizers, noise removal) manipulates signals by directly modifying their frequency content.

This shows that the choice of interpolation method has consequences far beyond pure mathematics: it underlies the modern digital world.

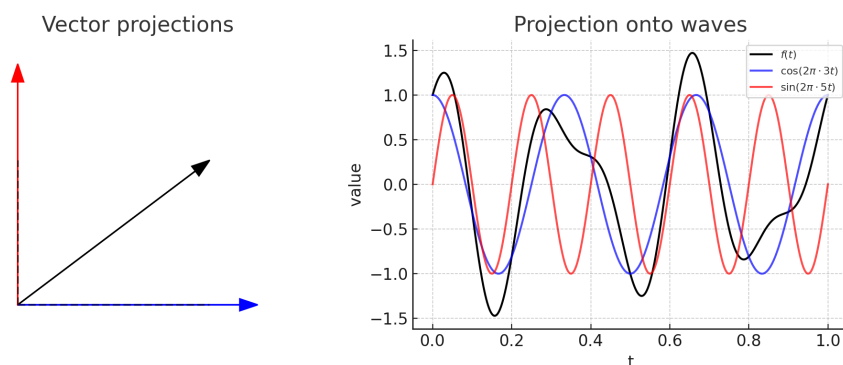
## 2 Fourier Series Basics

### 2.1 Why sines and cosines?

On a periodic interval like  $[0, 1)$ , sines and cosines are the natural building blocks: they repeat perfectly, they do not blow up at the “edges”, and—crucially—they do not *interfere* with one another when averaged over a full period. This lack of interference (orthogonality) is what makes them act like a coordinate system (**or a basis**) for periodic signals, just as  $(x, y, z)$  coordinates describe points in 3D.

#### Note

**Analogy.** In linear algebra, we project a vector on orthogonal axes to get coordinates. In Fourier analysis, we project a periodic function on orthogonal waves  $\cos(2\pi kt)$  and  $\sin(2\pi kt)$  to get *frequency coordinates*.



### 2.2 Inner product and orthogonality (clearer view)

For vectors in  $\mathbb{R}^2$ , the dot product  $\langle u, v \rangle = \|u\| \|v\| \cos \theta$  measures how much  $u$  is *aligned* with  $v$ . If the vectors are perpendicular, the dot product is 0.

For periodic functions we define the inner product

$$\langle f, g \rangle = \int_0^1 f(t) g(t) dt.$$

This measures how much  $f$  and  $g$  overlap over one full period. If their peaks and valleys cancel each other out on average, the inner product is 0.

With the inner product  $\langle f, g \rangle = \int_0^1 f(t) g(t) dt$  we have, for  $k, \ell \geq 1$ :

$$\int_0^1 \cos(2\pi kt) \cos(2\pi \ell t) dt = \begin{cases} 0, & k \neq \ell, \\ \frac{1}{2}, & k = \ell, \end{cases}$$

$$\int_0^1 \sin(2\pi kt) \sin(2\pi \ell t) dt = \begin{cases} 0, & k \neq \ell, \\ \frac{1}{2}, & k = \ell, \end{cases}$$

$$\int_0^1 \cos(2\pi kt) \sin(2\pi \ell t) dt = 0 \quad \text{for all } k, \ell \geq 1.$$

### Note

**Intuition.** Two different frequencies are like perpendicular directions in an abstract space. Over a full cycle their ups and downs cancel, so they carry no information about each other. This independence means we can measure each frequency's contribution separately, without contamination.

## 2.3 Definition: Fourier series and coefficients (real form)

So far we motivated that trigonometric polynomials are the natural building blocks for periodic functions. Now we make this precise.

**Step 1: Periodicity.** Let  $f : [0, 1] \rightarrow \mathbb{R}$  be a function with  $f(0) = f(1)$ . We imagine it extended periodically to all real  $t$ , so that it repeats every unit interval.

**Step 2: Goal.** We want to write  $f$  as a sum of pure waves:

$$f(t) \sim \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(2\pi kt) + b_k \sin(2\pi kt)).$$

-  $a_0/2$  is the average (the “DC component” or baseline level).  
 - Each term  $a_k \cos(2\pi kt)$  or  $b_k \sin(2\pi kt)$  captures how much  $f$  oscillates at frequency  $k$ .  
 - Taking the sum over  $k$  combines these oscillations to approximate or reconstruct  $f$ .

**Step 3: How to find the coefficients.** We use the orthogonality of the basis waves. Projecting  $f$  onto  $\cos(2\pi kt)$  isolates the  $a_k$  coefficient, and projecting onto  $\sin(2\pi kt)$  isolates the  $b_k$  coefficient:

$$a_0 = 2 \int_0^1 f(t) dt, \quad a_k = 2 \int_0^1 f(t) \cos(2\pi kt) dt, \quad b_k = 2 \int_0^1 f(t) \sin(2\pi kt) dt.$$

These integrals act like “measuring glasses”: they tell us exactly how much of each basic wave is present inside  $f$ .

### Note

**Physical analogy.** Think of shining white light through a prism: the light splits into distinct colors (frequencies). The integrals above are the mathematical prism: they split a function into pure cosine and sine components.

**Step 4: Interpretation.** -  $a_0$  is proportional to the average value of  $f$ . -  $a_k$  measures how strongly  $f$  correlates with a cosine wave of frequency  $k$ . -  $b_k$  measures how strongly  $f$  correlates

with a sine wave of frequency  $k$ .

In short:  $a_k$  and  $b_k$  are the *coordinates of  $f$  in frequency space*.

## 2.4 Complex form of the Fourier series

So far we wrote Fourier series in the *real form*:

$$f(t) \sim \frac{a_0}{2} + \sum_{k=1}^{\infty} \left( a_k \cos(2\pi kt) + b_k \sin(2\pi kt) \right).$$

**Step 1: A new notation.** Introduce the basic complex exponentials

$$e_k(t) := e^{2\pi i k t}, \quad k \in \mathbb{Z}.$$

Each  $e_k$  is itself periodic with period 1, and by Euler's formula

$$e^{i\theta} = \cos(\theta) + i \sin(\theta).$$

So these exponentials automatically encode both cosines and sines.

**Step 2: Fourier series in complex form.** Using  $e_k(t)$ , the Fourier series of  $f$  can be written as

$$f(t) \sim \sum_{k=-\infty}^{\infty} c_k e^{2\pi i k t},$$

where the complex coefficients are given by the projection formula

$$c_k = \int_0^1 f(t) e^{-2\pi i k t} dt.$$

**Step 3: Relation to the real form.** The two notations are completely equivalent:

$$c_0 = \frac{a_0}{2}, \quad c_k = \frac{a_k - i b_k}{2} \quad (k \geq 1), \quad c_{-k} = \overline{c_k}.$$

That is, the real coefficients  $(a_k, b_k)$  can be combined into a single complex number  $c_k$ .

**Step 4: Why use the complex form?**

- Algebra becomes shorter: products and derivatives of exponentials are simpler than juggling cosines and sines separately.
- Symmetry: the negative indices  $-k$  capture the sine components automatically, and the relation  $c_{-k} = \overline{c_k}$  ensures the series is real-valued.
- In practice: most proofs, algorithms (like the FFT), and applications in physics/engineering are expressed using the complex form.

**Note**

**Interpretation.** - In the real form:  $a_k$  and  $b_k$  are like two separate coordinates (cosine-part and sine-part). - In the complex form:  $c_k$  is a single coordinate in the complex plane, encoding both magnitude and phase of frequency  $k$ . This makes the frequency representation much more compact and elegant.

### 3 Discrete Fourier Transform (DFT)

#### 3.1 From continuous to discrete

Up to now we have seen how to expand a *continuous periodic function* into a Fourier series. But in practice we never know  $f(t)$  everywhere: we only measure it at discrete sample points

$$t_j = \frac{j}{N}, \quad j = 0, 1, \dots, N-1,$$

with values

$$y_j = f(t_j).$$

#### Note

**Bridge.** Fourier series answer the question: “If I know the function exactly, what are its frequency coefficients?” The Discrete Fourier Transform (DFT) answers the practical question: “If I only know  $N$  sampled values, how do I extract frequency information from them?”

#### 3.2 Trigonometric interpolation viewpoint

Recall: with  $N$  samples we can build a trigonometric polynomial of degree at most

$$m = \left\lfloor \frac{N-1}{2} \right\rfloor$$

that interpolates all samples exactly. This interpolant has the form

$$p(t) = \sum_{k=-m}^m c_k e^{2\pi i k t}.$$

The unknowns are the coefficients  $c_k$ , and the equations are the interpolation conditions

$$p(t_j) = y_j, \quad j = 0, 1, \dots, N-1.$$

Thus, computing the  $c_k$  from the  $y_j$  is precisely what the DFT does.

#### 3.3 Roots of unity and orthogonality

The key algebraic building block of the DFT is the  $N$ -th root of unity:

$$\omega_N = e^{-2\pi i/N}.$$

**Step 1: What are they?** If we take successive powers of  $\omega_N$ , we obtain

$$1, \omega_N, \omega_N^2, \dots, \omega_N^{N-1}.$$

These  $N$  points lie equally spaced on the unit circle in the complex plane, at angles

$$0, -\frac{2\pi}{N}, -\frac{4\pi}{N}, \dots, -\frac{2\pi(N-1)}{N}.$$

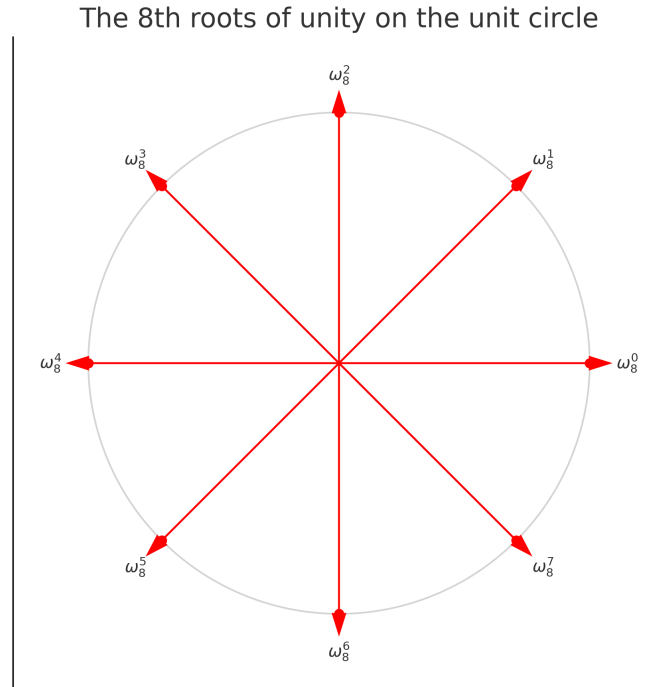


Figure 4: The  $N$ -th roots of unity for  $N = 8$ : equally spaced points on the unit circle, forming a regular octagon. The symmetry explains why their sum is zero (orthogonality).

**Step 2: Why are they useful?** Imagine drawing arrows (vectors) from the origin to each of these points. Because they are symmetrically placed, the arrows cancel when summed:

$$1 + \omega_N + \omega_N^2 + \dots + \omega_N^{N-1} = 0.$$

This is true whenever you go once around the circle (i.e. for any nontrivial rotation).

**Step 3: The orthogonality relation.** More generally, if we take

$$\sum_{j=0}^{N-1} \omega_N^{jk},$$

then

$$\sum_{j=0}^{N-1} \omega_N^{jk} = \begin{cases} N, & k \equiv 0 \pmod{N}, \\ 0, & \text{otherwise.} \end{cases}$$



## Note

**Intuition.** - Think of walking around the unit circle  $N$  steps at a time. - If  $k = 0$ , you never rotate: all vectors are 1, so their sum is  $N$ . - If  $k \neq 0$ , you rotate by an exact fraction of the circle: the vectors distribute evenly, forming a regular polygon. The polygon is perfectly symmetric, so the sum of its vertices is 0.

This cancellation is the geometric meaning of *orthogonality*. It ensures that each frequency component is independent of the others, just like perpendicular directions in Euclidean space.

### 3.4 Fourier matrix and the DFT

We can arrange the complex exponentials into a matrix called the *Fourier matrix*:

$$F_N(i, j) = \omega_N^{ij}, \quad i, j = 0, \dots, N-1,$$

where  $\omega_N = e^{-2\pi i/N}$  is the primitive  $N$ -th root of unity.

**The Fourier matrix explicitly.** For  $N = 4$ , the primitive root of unity is

$$\omega_4 = e^{-2\pi i/4} = e^{-i\pi/2} = -i.$$

The Fourier matrix is

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega_4 & \omega_4^2 & \omega_4^3 \\ 1 & \omega_4^2 & \omega_4^4 & \omega_4^6 \\ 1 & \omega_4^3 & \omega_4^6 & \omega_4^9 \end{bmatrix}.$$

Simplifying the powers of  $\omega_4$  gives

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}.$$

In general, for arbitrary  $N$ , the Fourier matrix is

$$F_N = \begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_N & \omega_N^2 & \dots & \omega_N^{N-1} \\ 1 & \omega_N^2 & \omega_N^4 & \dots & \omega_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \omega_N^{2(N-1)} & \dots & \omega_N^{(N-1)(N-1)} \end{bmatrix}.$$

## Note

**Columns as a basis of  $\mathbb{C}^N$ .**

Each column of  $F_N$  is a vector

$$v_k = (1, \omega_N^k, \omega_N^{2k}, \dots, \omega_N^{(N-1)k})^T, \quad k = 0, 1, \dots, N-1.$$

- These are the sampled exponentials  $e^{2\pi i k t_j}$  on the uniform grid  $t_j = j/N$ . - The orthogonality of roots of unity means that  $\{v_0, \dots, v_{N-1}\}$  are mutually orthogonal vectors in  $\mathbb{C}^N$ . - After scaling by  $1/\sqrt{N}$ , they form an *orthonormal basis*.

So the DFT is nothing more than a *change of basis* from the standard basis to the Fourier basis.

**What does this mean?** - Each column of  $F_N$  corresponds to one frequency  $k$ . - Each row corresponds to one sample  $t_j = j/N$ . - The entry  $\omega_N^{ij}$  is exactly  $e^{-2\pi i k t_j}$ : the value of frequency  $k$  at sample  $j$ .

So  $F_N$  is like a big “lookup table” of how every frequency looks on the sampling grid.

**Forward DFT.** Multiplying by  $F_N$  projects the data onto these frequencies:

$$c = F_N y,$$

where -  $y = (y_0, \dots, y_{N-1})^T$  are the samples, -  $c = (c_0, \dots, c_{N-1})^T$  are the Fourier coefficients (how much of each frequency is present).

**Inverse DFT.** Because the basis waves are orthogonal, the process is invertible. The inverse is

$$y = \frac{1}{N} F_N^H c,$$

where  $F_N^H$  is the conjugate transpose of  $F_N$ .

This means we can go back from frequencies to the original samples without losing information.

## Note

**Big picture.** The DFT is just a change of coordinates:

- In the *time domain* the signal is described by its samples  $y_j$  at different times.
- In the *frequency domain* the same signal is described by coefficients  $c_k$  attached to different pure frequencies.

Because the Fourier basis functions are orthogonal (they don't interfere), this change of coordinates is stable and invertible. It is exactly like switching from Cartesian to polar coordinates in geometry: the same point, just described in another basis.

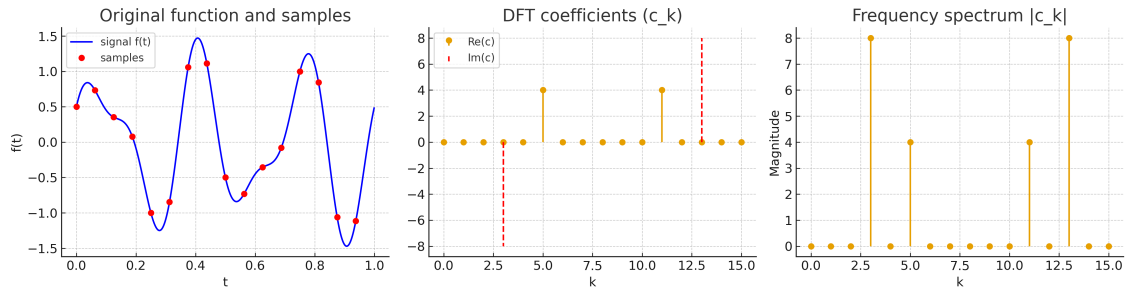


Figure 5: Pipeline: periodic function (top-left)  $\rightarrow$  uniform samples  $y_j \rightarrow$  DFT coefficients  $c_k$  (top-right)  $\rightarrow$  frequency-domain magnitudes  $|c_k|$  (bottom).

### 3.5 Properties and impact

- **Energy preservation.** The Fourier matrix is unitary up to scaling, so

$$\sum_{j=0}^{N-1} |y_j|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |c_k|^2.$$

This is the discrete analogue of Parseval's theorem.

- **Computation: FFT.** A naive DFT costs  $O(N^2)$  operations. The *Fast Fourier Transform (FFT)* is a clever algorithm that reduces this to  $O(N \log N)$ . This efficiency is what makes real-time audio, video, and scientific computing possible.

### 3.6 Why is the DFT useful?

The Discrete Fourier Transform is not only a piece of linear algebra. It is the mathematical backbone of almost every digital technology: audio, images, video, communication, even medical scans.

**Example: digital audio.** When you speak into your phone:

1. The microphone converts sound waves into a continuous electrical signal  $f(t)$ .
2. The signal is **sampled** at a high rate (e.g. 8–16 kHz for phone calls, 44.1 kHz for music).
3. The DFT is applied to short windows of these samples to measure how much energy each frequency contains.
4. The frequency information can be:
  - stored (MP3, AAC compression keeps only the important frequencies for the ear),
  - transmitted (cellular networks send compact frequency-domain data),
  - or manipulated (noise cancellation, equalizers).
5. On playback, the inverse DFT reconstructs the sound wave from its frequency representation.

**Why do we sample?** A real sound wave  $f(t)$  is a continuous function of time. Even in just *one second* of recording,  $t$  takes uncountably many values. Storing all of them would require infinite memory.

Instead, digital systems use two key ideas:

1. **Sampling:** We only keep values of  $f(t)$  at equally spaced times  $t_j = j/N$ . For audio,  $N$  is large but finite (e.g. 44,100 samples per second for CD quality).
2. **Fourier analysis:** We apply the DFT to those samples. This transforms the time sequence  $y_j$  into a frequency representation  $c_k$ . Instead of storing every wiggle of the waveform, we just store how much of each sine/cosine wave is present.

#### Note

**Big picture.** Continuous audio cannot be stored directly — it would require infinitely many data points even for one second. By combining *sampling* (make it finite) with the *DFT* (make it interpretable as frequencies), we obtain a practical representation. This is why your phone can record, compress, transmit, and replay sound in real time.

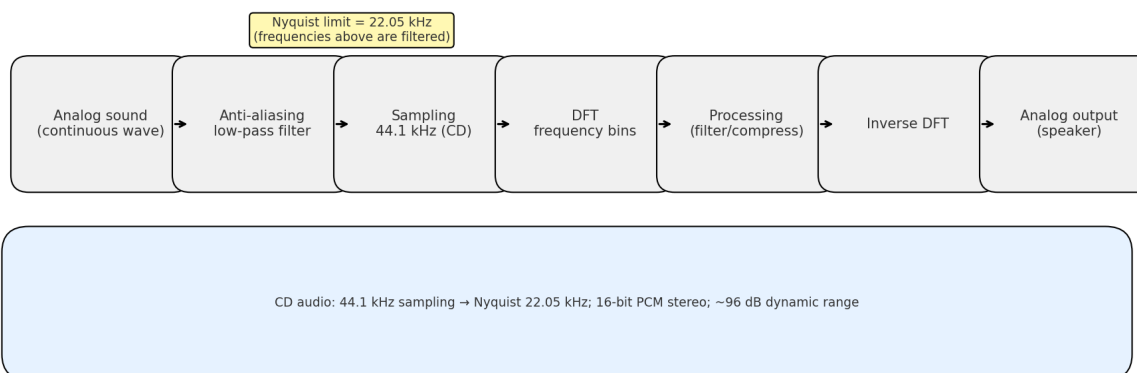


Figure 6: Audio pipeline: analog sound is sampled (44.1 kHz for CD), transformed via DFT into frequency bins, processed (filtering/compression), then reconstructed back to analog output. Nyquist limit at 22.05 kHz ensures no aliasing in the audible range.